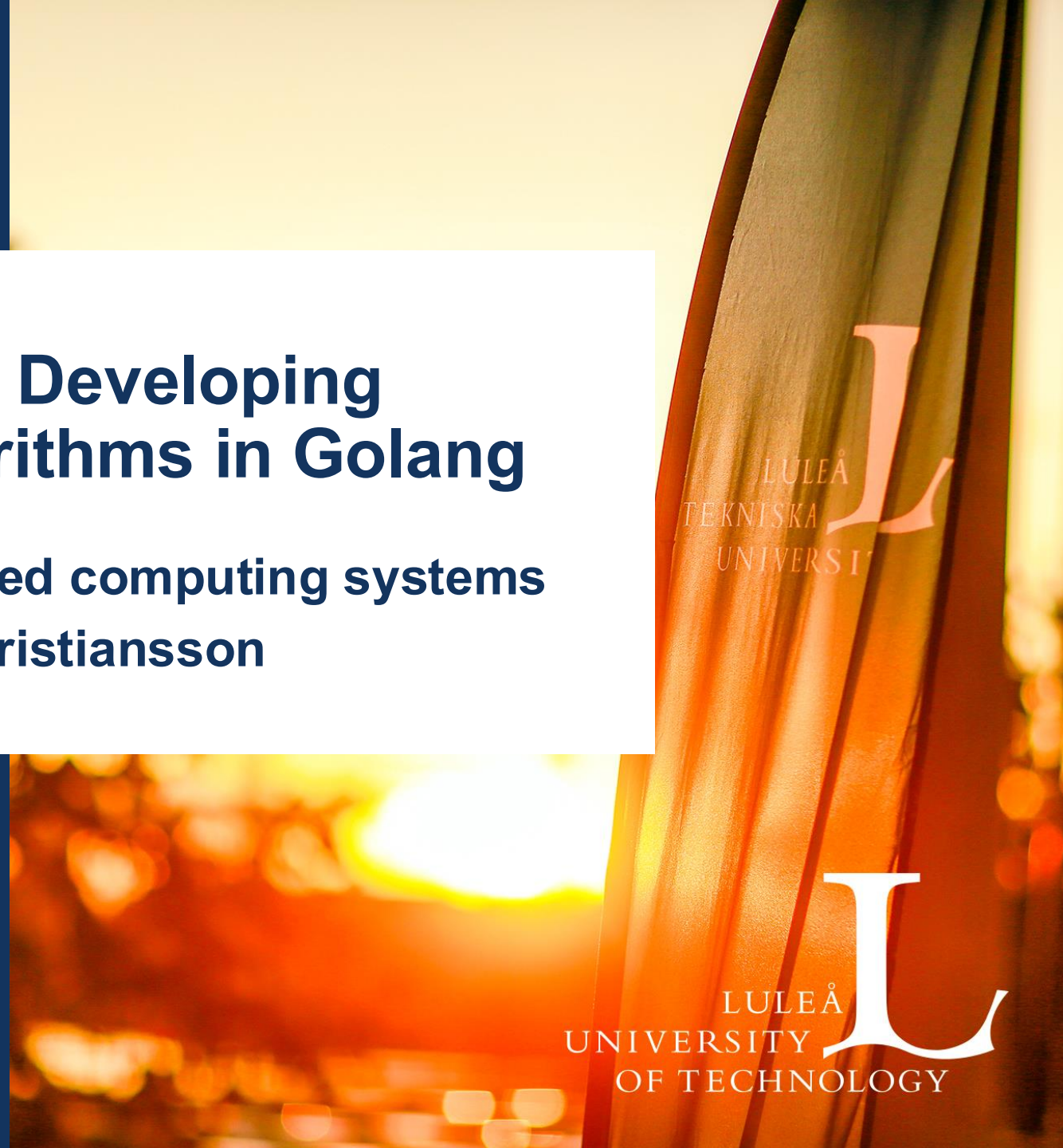
Practical lesson: Developing Distributed Algorithms in Golang

Mobile and distributed computing systems

Johan Kristiansson



Principal Researcher
Ericsson AB
2006 – 2017

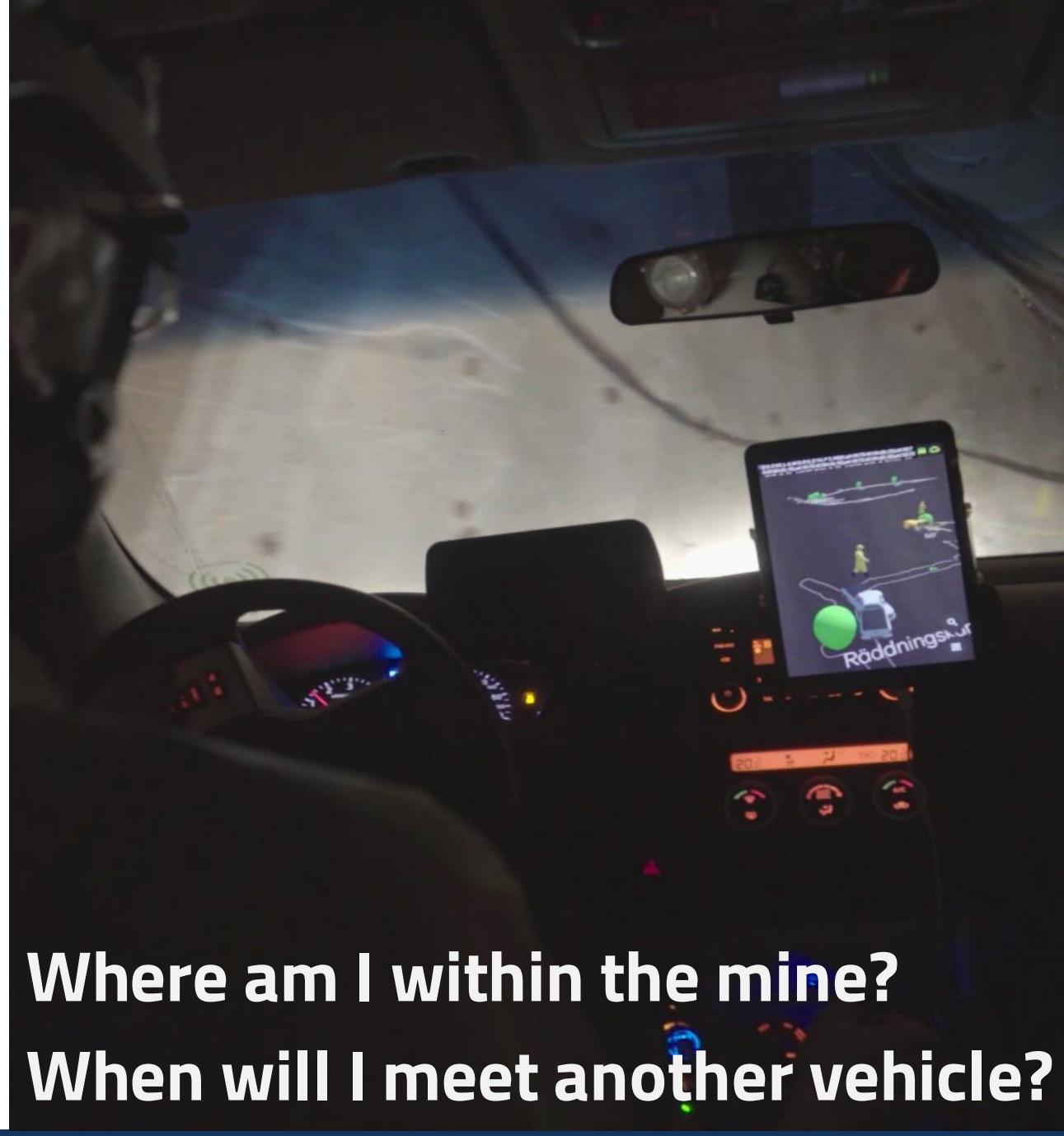


Investigate distruptive technologies **(e.g social media, web browsers, blockchains)** and impact on tranditional telecommunication services **(e.g. connectivity, phone/SMS)**?

Data Scientist & Technical Lead
Mobilaris AB
2017 – 2019

Mobilaris Onboard





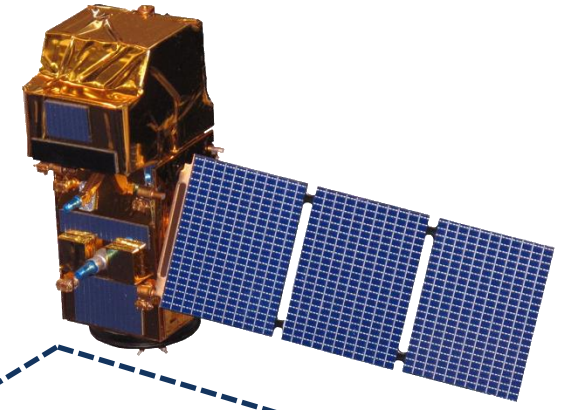

BOLIDEN







Senior Researcher
RISE AB
2019 – 2025



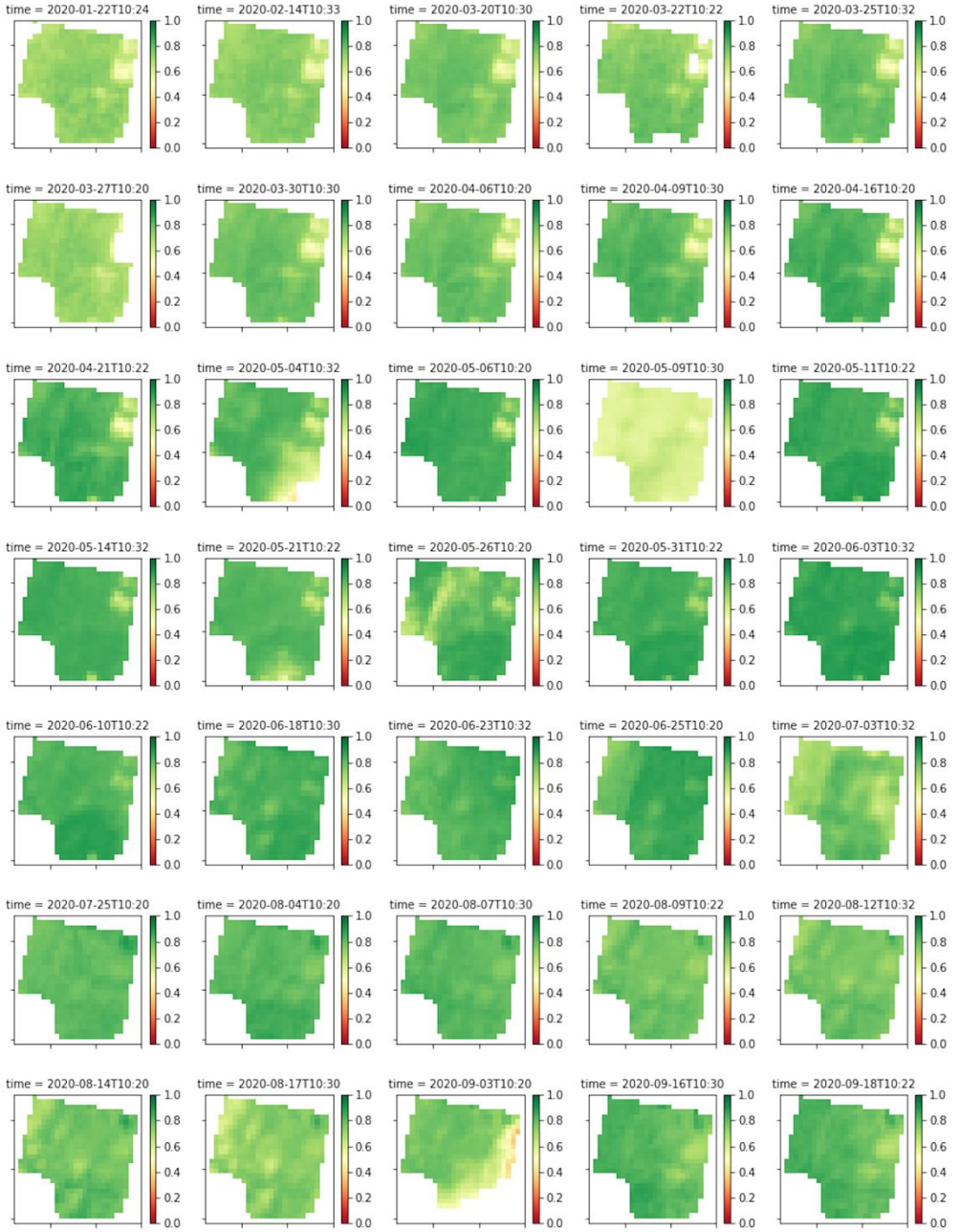
Did the cow graze?

Jordbruks verket

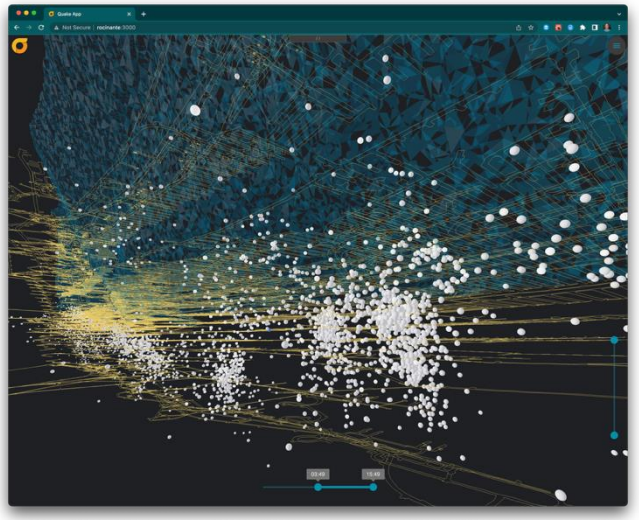


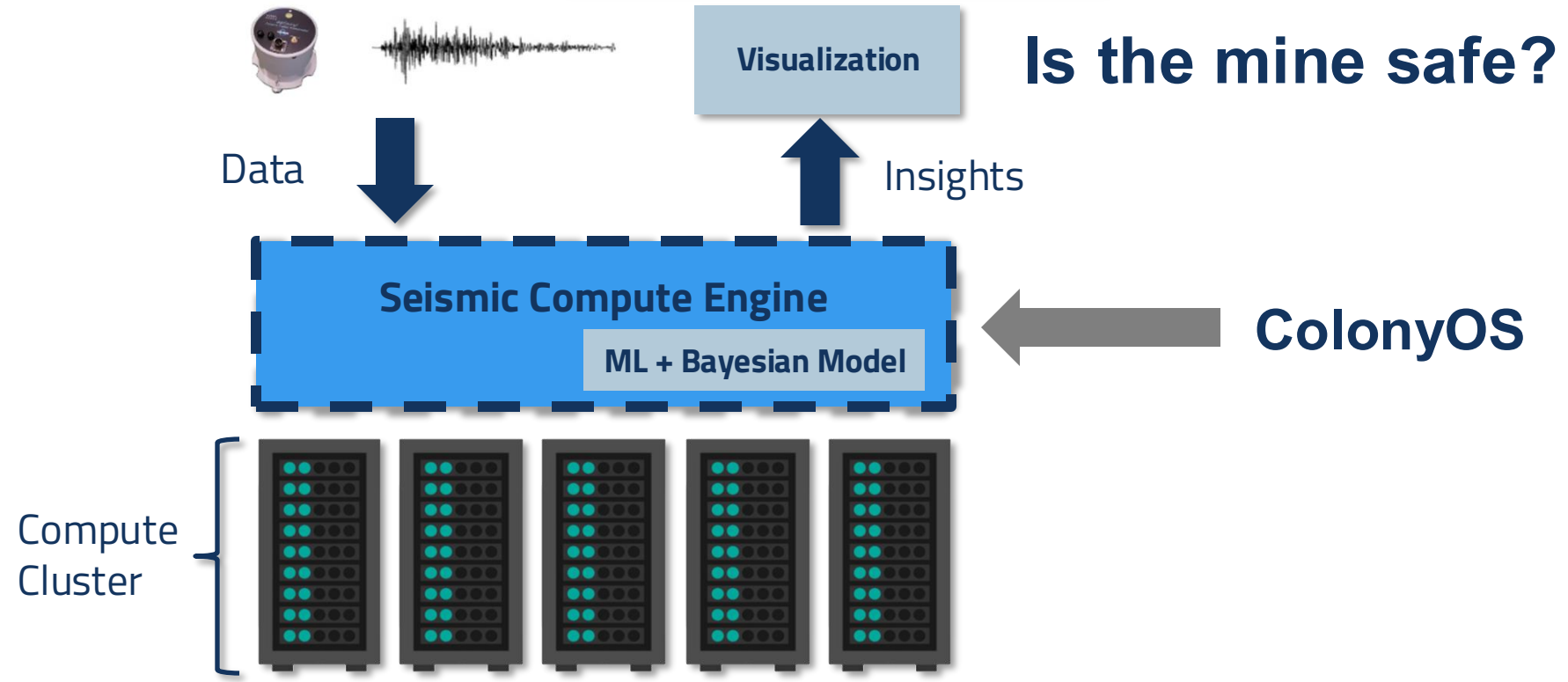
SKOGSSTYRELSEN

Tell me when this forest is harvested?



Chief Technology Officer
RockSigma AB
2022 – 2023





Senior Researcher
RISE AB
2019 – now

- Supercomputers (HPC) + AI
- Carbon-aware computing
- Edge computing
- **Compute Continuum**



Large-Language Models



ENCCS
EuroCC National Competence Centre Sweden

European enterprises

ColonyOS

"Modern" AI Toolkits

Workload Manager

HPC

GPUs

LUMI:
5026 nodes
11912 GPUs

GPUs

ColonyOS


colonyos.io
ColonyOS
A Meta-OS for Computing Continuums
GitHub
What is ColonyOS?
In an increasingly digitalized world, seamless interoperability and scalable, high-performance computing are not optional—they are fundamental. As we move towards more decentralized and heterogeneous computing infrastructures, the complexities inherent in developing cross-platform applications grow significantly. Envision a future where AI workloads can be developed and executed seamlessly across diverse environments, including IoT, Edge, Cloud, and HPC systems.
ColonyOS is an open-source research project currently developed by Luleå University of Technology, RISE AB. It is used by ENCCS to foster greater High-Performance Computing (HPC) adoption. It is also used by RockSigma AB to implement a compute engine designed for seismic processing in underground mines. RockSigma AB has contributed to the development of ColonyOS.
Links
Tutorials
Contact us
Use Cases
Distributed Compute Engines
Implement distributed compute engines that optimize data processing across diverse platforms. Perform intensive computations on one platform and then effortlessly merge the results with an external business system. This is particularly useful in many remote sensing applications (such as seismic processing), which require both heavy computations and system integration.
Streamlined HPC
ColonyOS offers modern APIs and cloud integration, expanding supercomputers' reach and accessibility. HPC Executors enables easy, platform-agnostic deployment of workloads, boosting global collaboration and making HPC universally accessible across academia, industry, and beyond.
Virtual Supercomputing
Harness and combine computational power of multiple disparate computing systems, whether HPC, cloud-based infrastructures, or other computing resources, to function as a single, unified supercomputer. This virtual supercomputer can execute large-scale tasks, simulations, or calculations that would be beyond the capability of any single system within the combined network.
Compute Continuums
Compute Continuums refer to a unified computing framework spanning across cloud, edge, and IoT. By ensuring cohesive computing, computations can migrate seamlessly between centralized cloud servers, edge servers closer to data
Energy-aware Computing
With a brokering model, the process of executing computations is kept separate from actually obtaining compute resources. This offers flexibility. This can be used to set constraints on computations, for example only execute certain processes on
Digital Sovereignty
Maintain strict control and autonomy by executing computations without moving data. For instance, securely process medical data at a local hospital server and integrate less-sensitive results into external systems, all within one workflow.

Developing distributed algorithms in Golang

1 Go project foundation
- Best practices & modern project structure & organisation
- GitHub Actions CI/CD setup

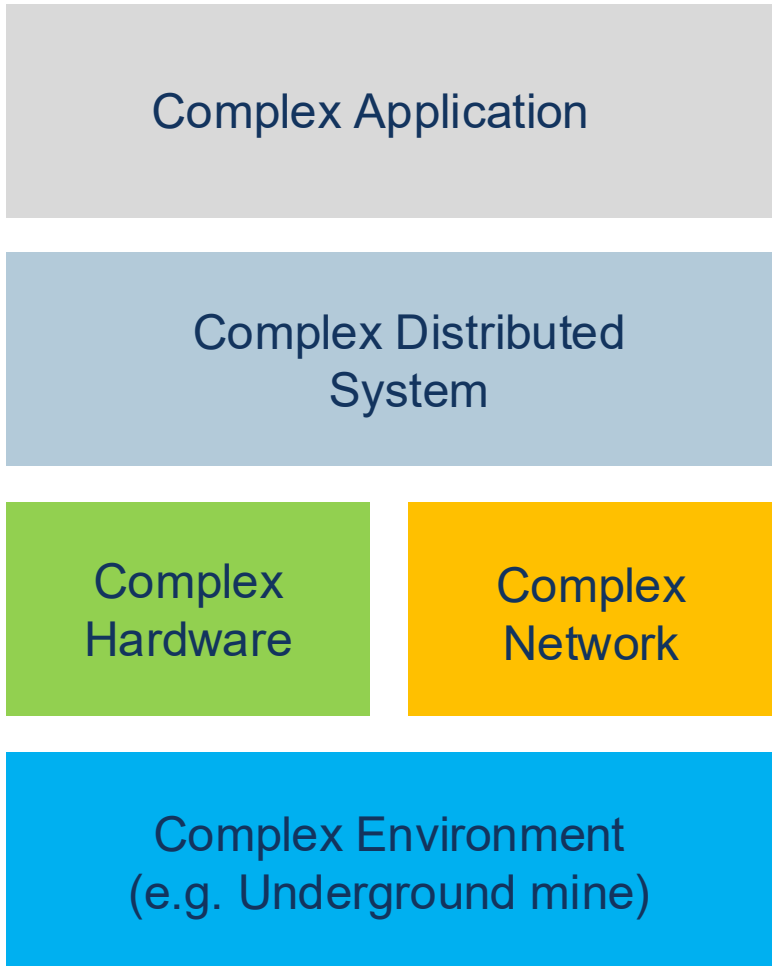
2. Building modern CLIs
- Clean architecture & separation of concerns
- Logging strategies
- Command-line interface design

3. Master Go concurrency
- Goroutines & channels deep dive
- Synchronization patterns
- Testing concurrent code

4. Learn how to mock and simulate networks

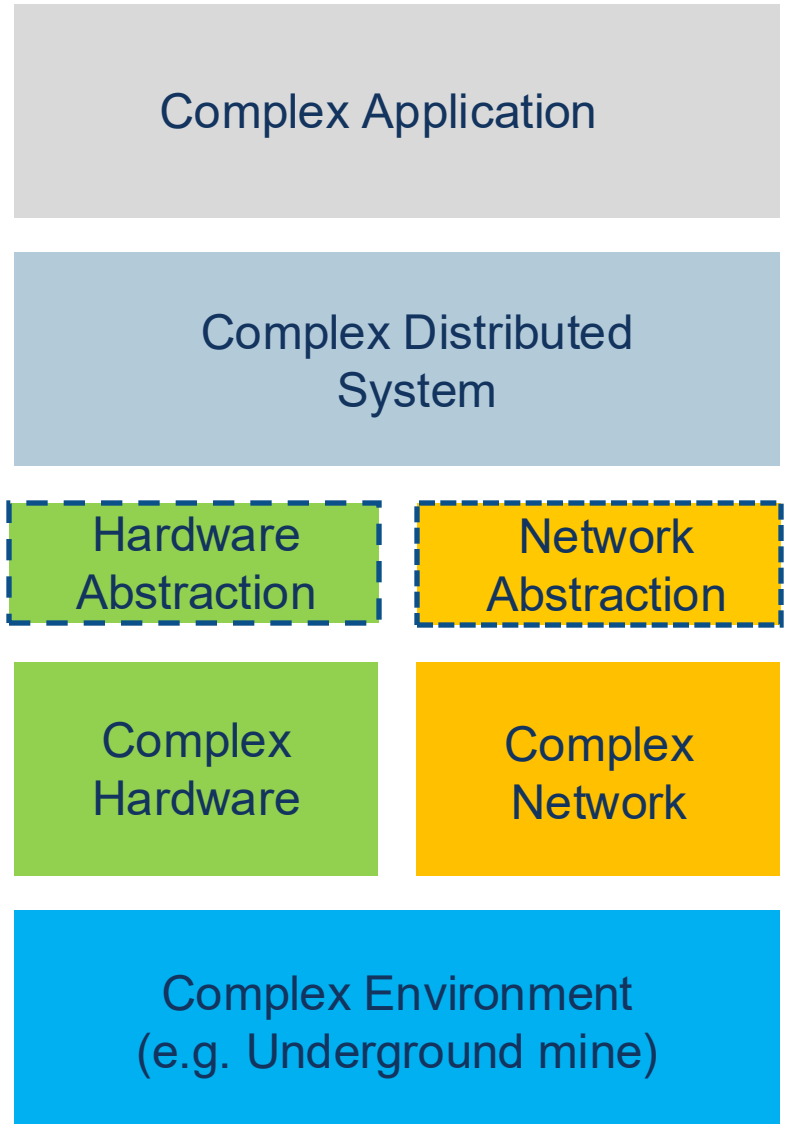
5. Implementing and testing a simple distributed algorithm

Engineering Principles

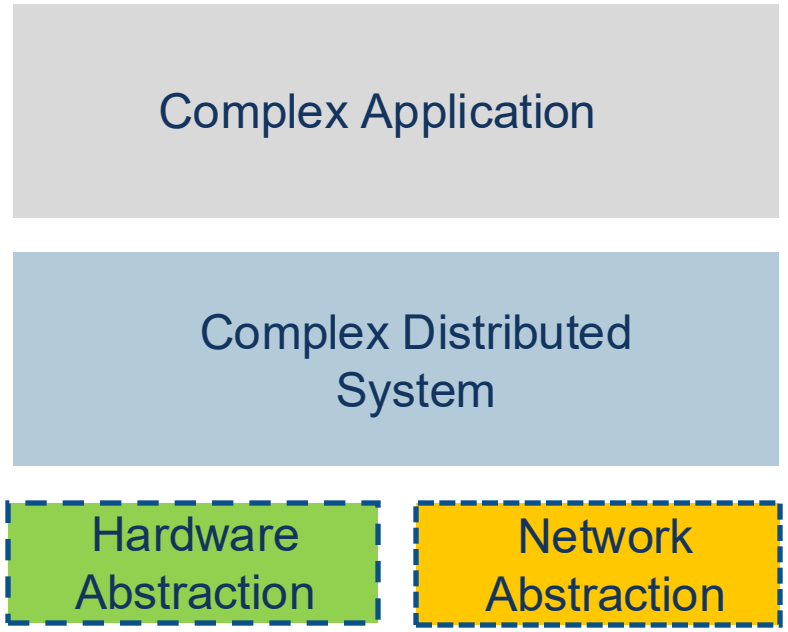


Engineering Principles

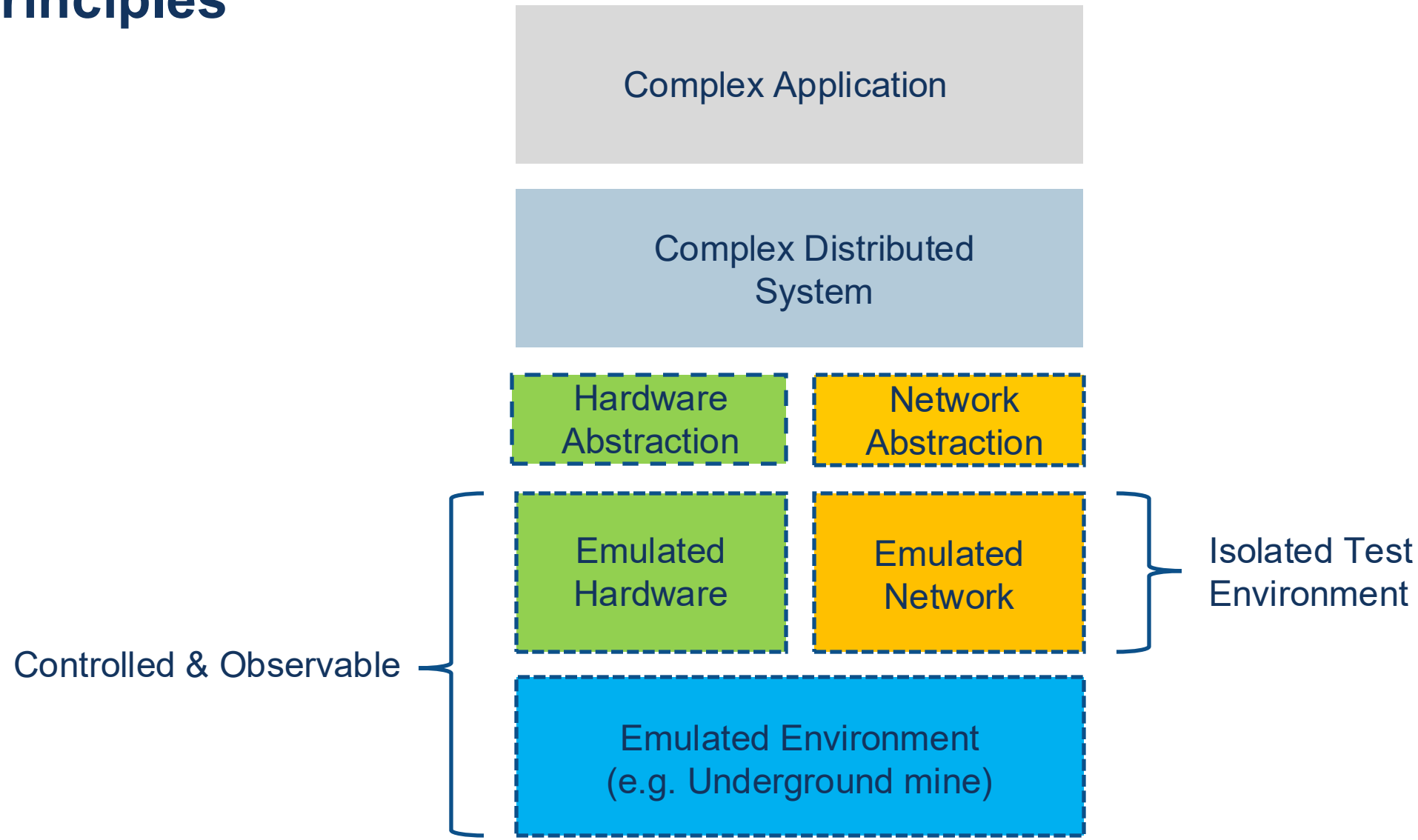
We can always add abstraction to hide and manage complexity?



Engineering Principles



Engineering Principles



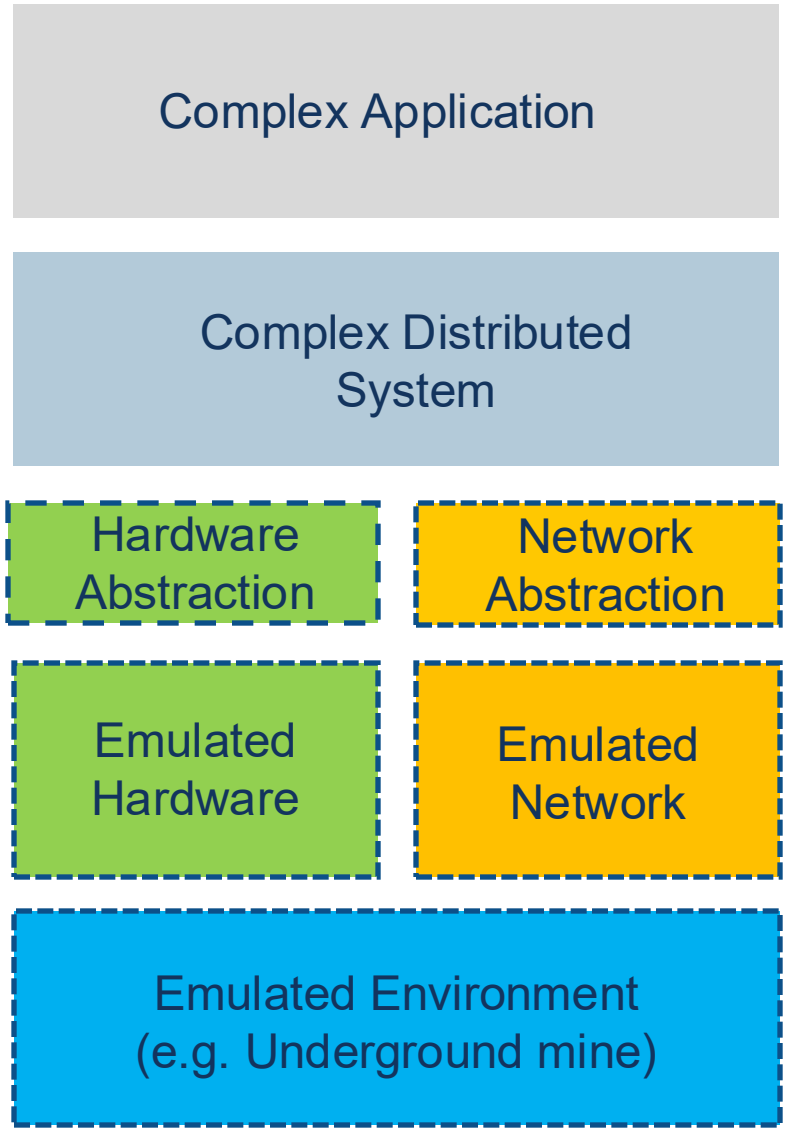
Engineering Principles

Faster Development Cycles

- Instant environment provisioning - spin up test environments in seconds, not hours
- Parallel testing - run 100 different network conditions simultaneously
- Time manipulation - simulate a week of operations in minutes
- Reproducible failures - capture and replay exact bug conditions

Risk Reduction

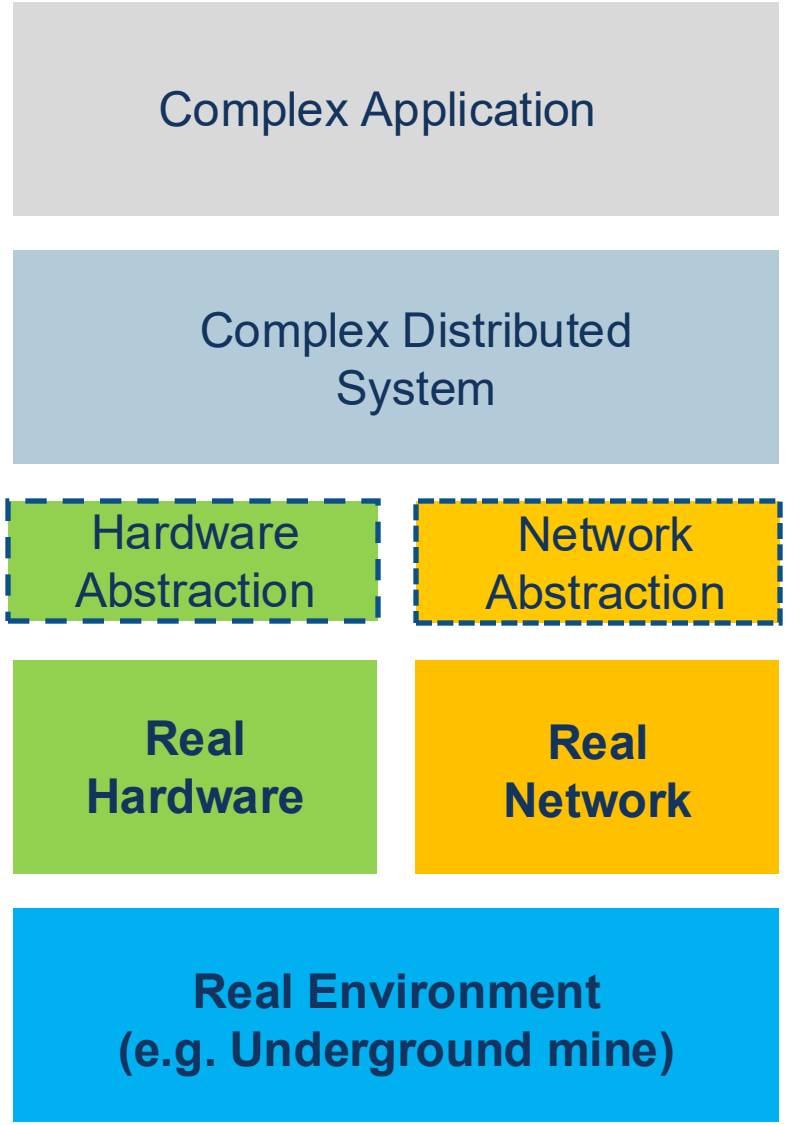
- Safe failure testing - simulate catastrophic scenarios (network partitions, hardware failures) without real consequences
- Edge case exploration - test conditions rare in production (99% packet loss, extreme latency)
- Rollback safety - know your software won't corrupt data or crash systems before touching real infrastructure



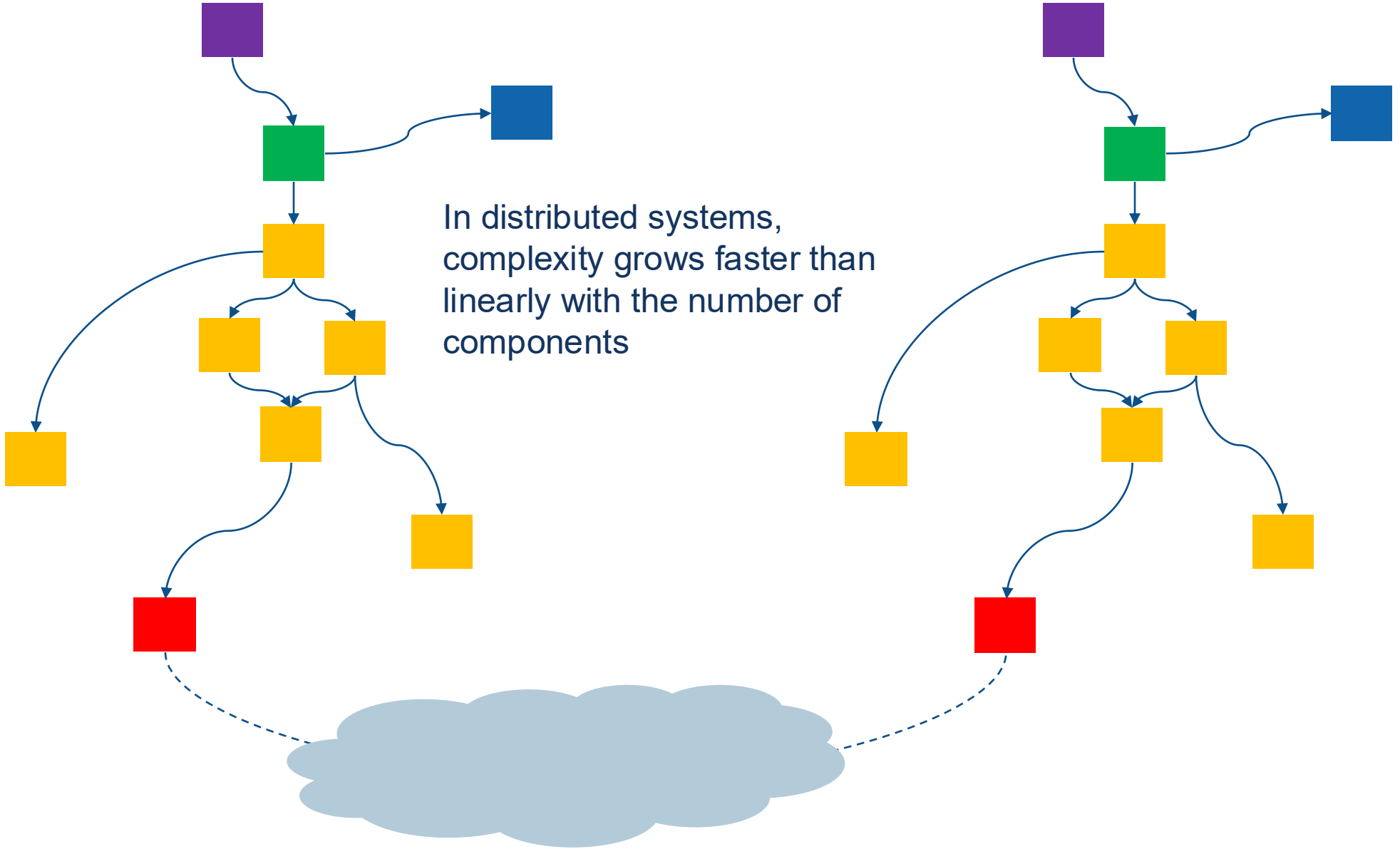
Engineering Principles

When SW works correctly in emulation, you've proven the logic and algorithms are sound, ***any future failures are likely environmental, not fundamental design flaws***. This separation of concerns is invaluable for debugging.

If it ***doesn't work*** in the perfect conditions of emulation, it definitely ***won't work*** in the chaos of production. If it ***does work*** in emulation, you've ***eliminated an entire class of potential failures***.



Testing

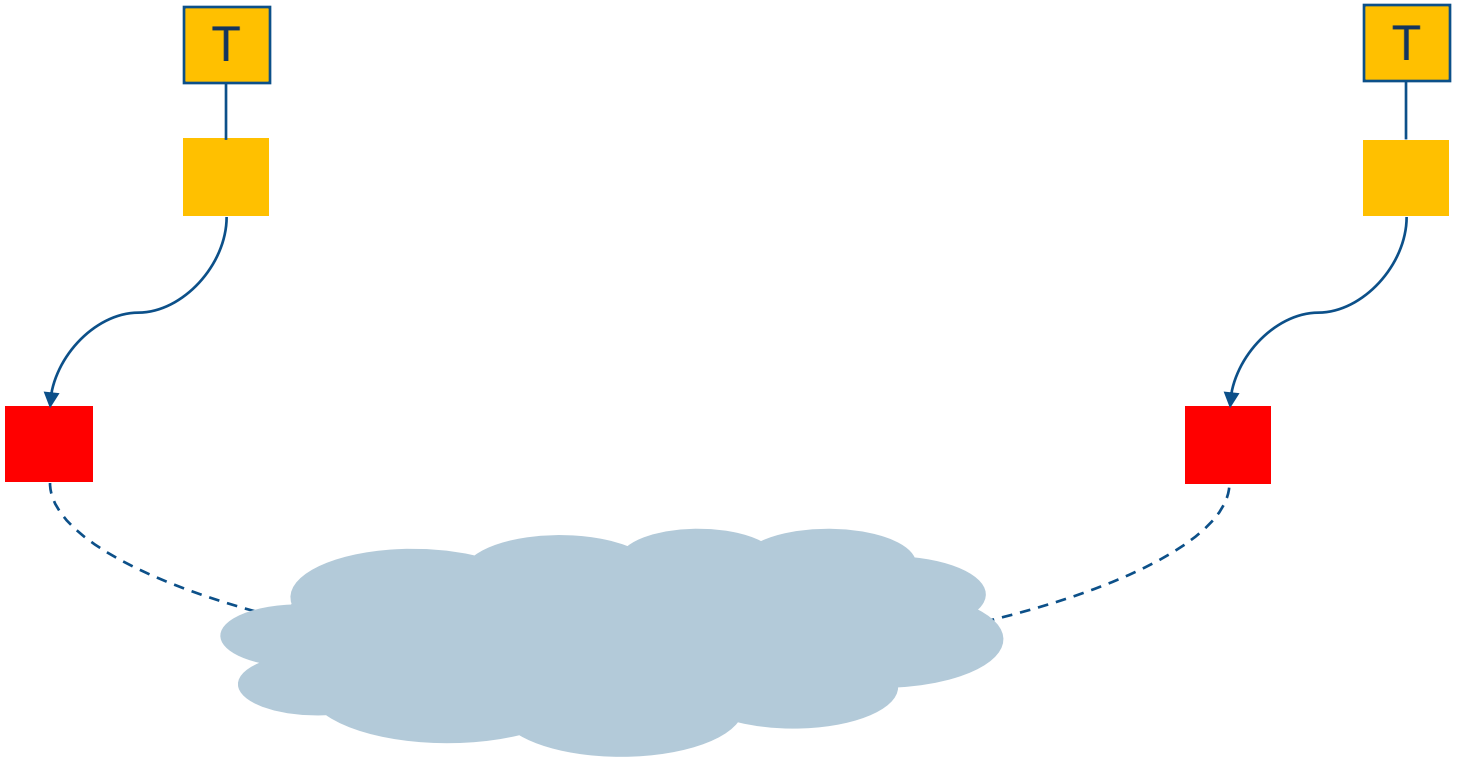


In distributed systems, complexity grows faster than linearly with the number of components

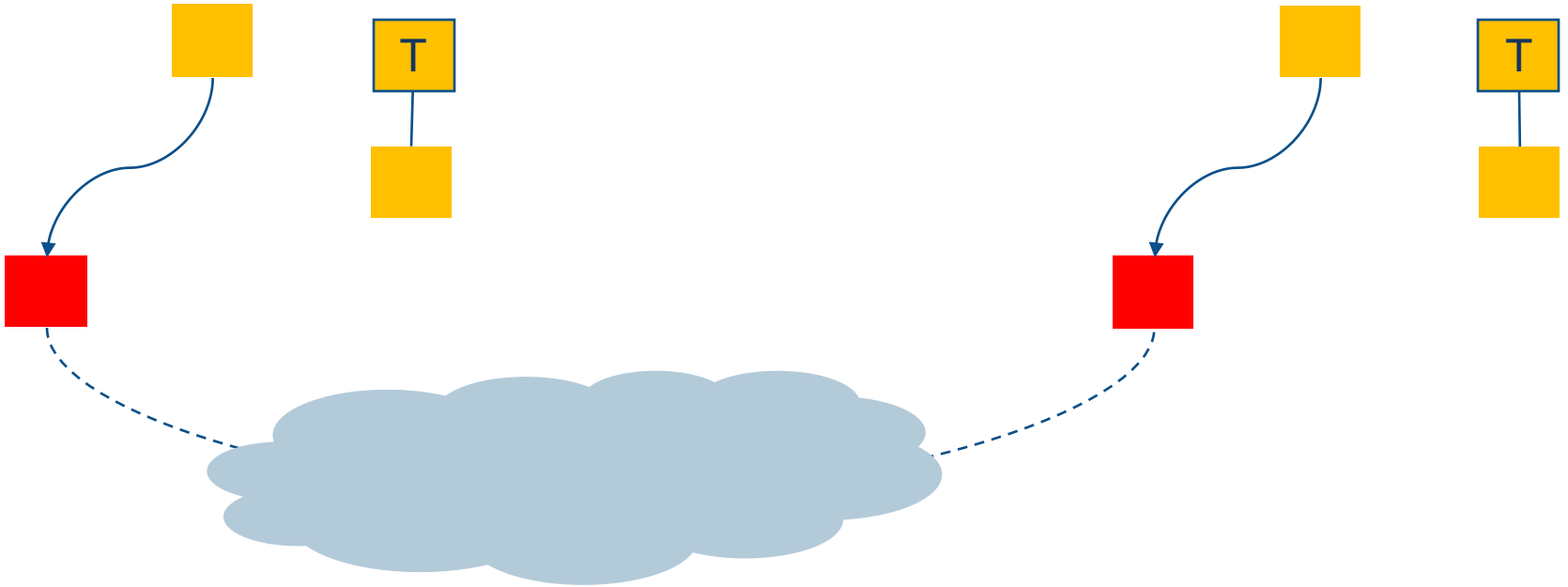
Testing



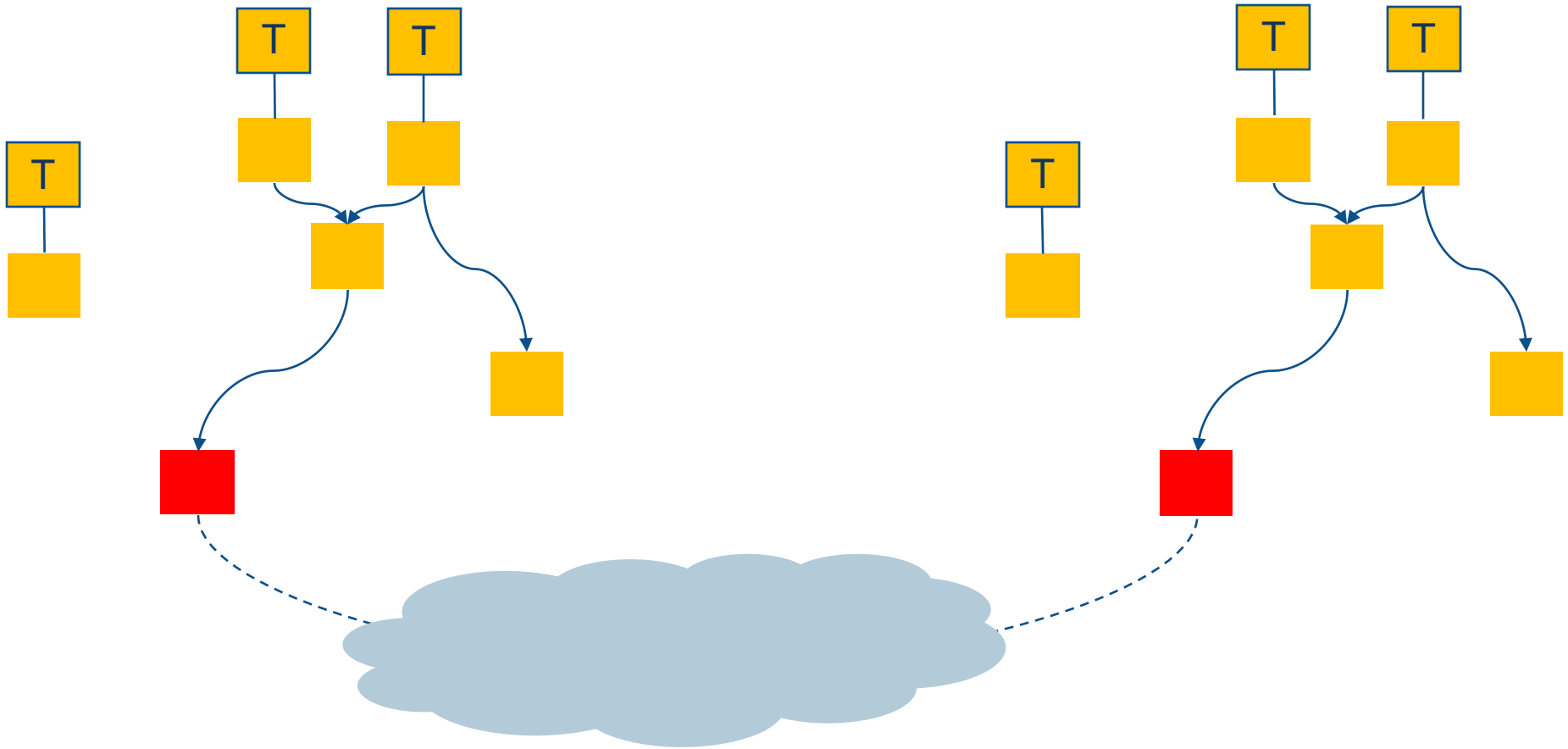
Testing



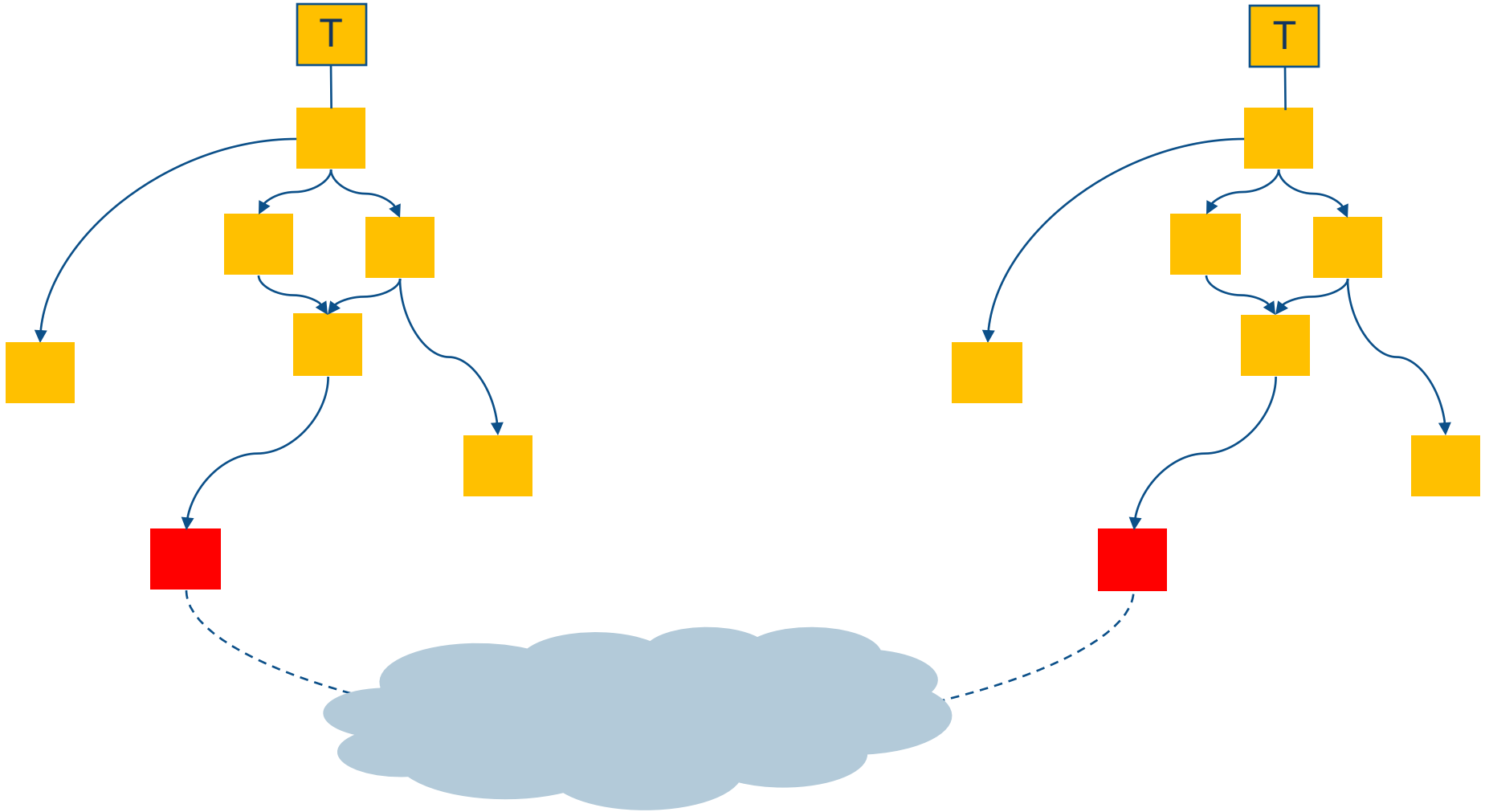
Testing



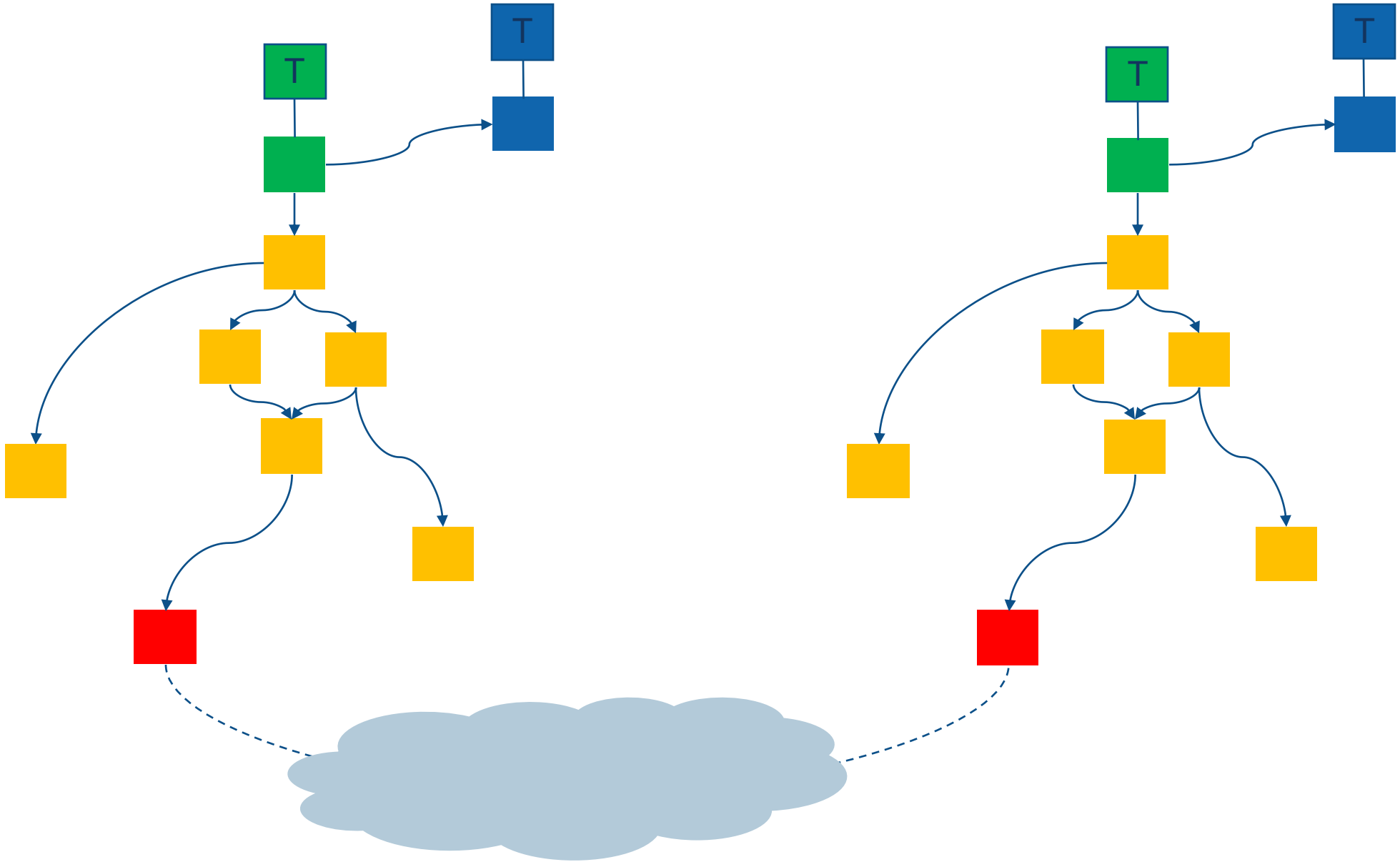
Testing



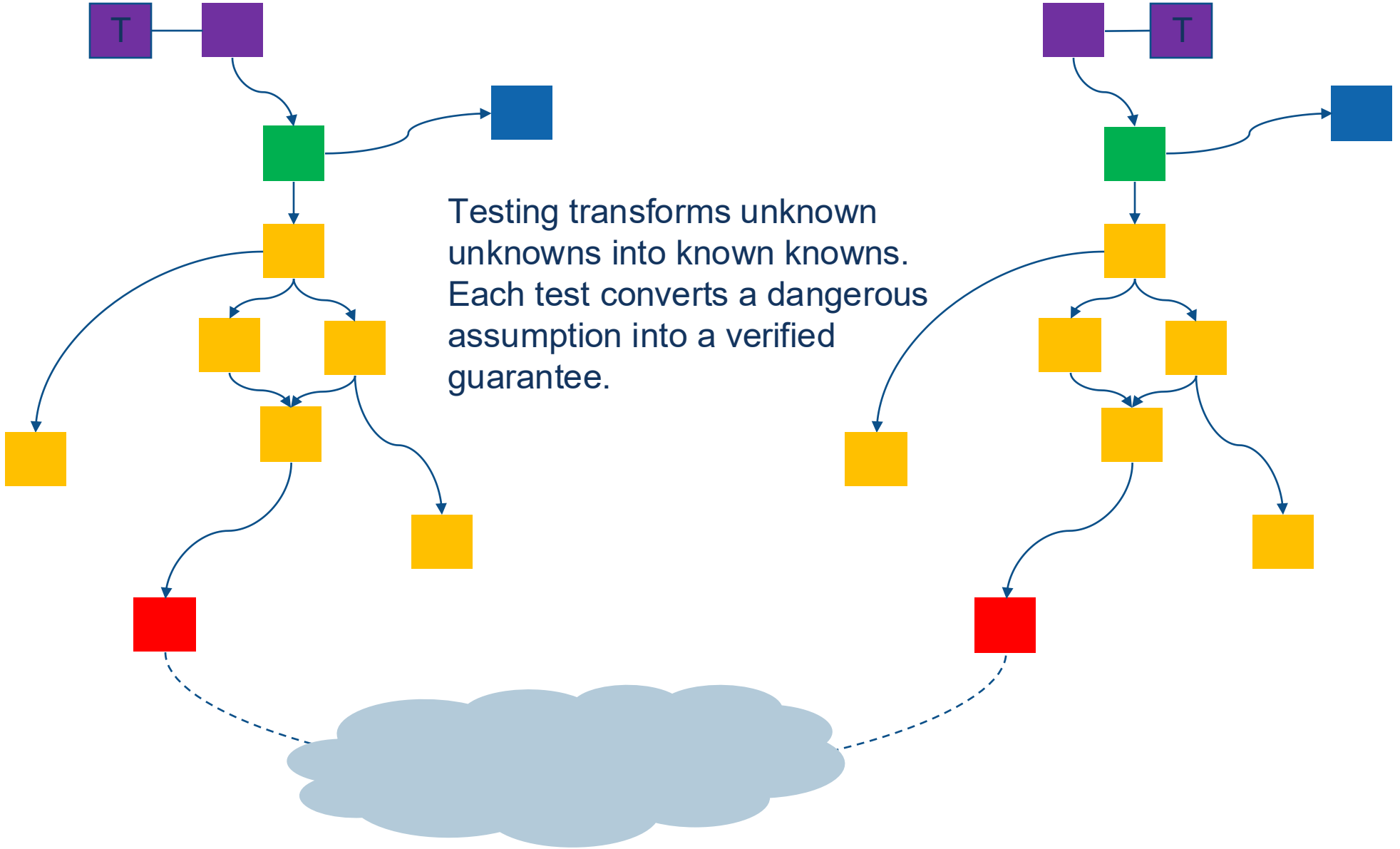
Testing



Testing

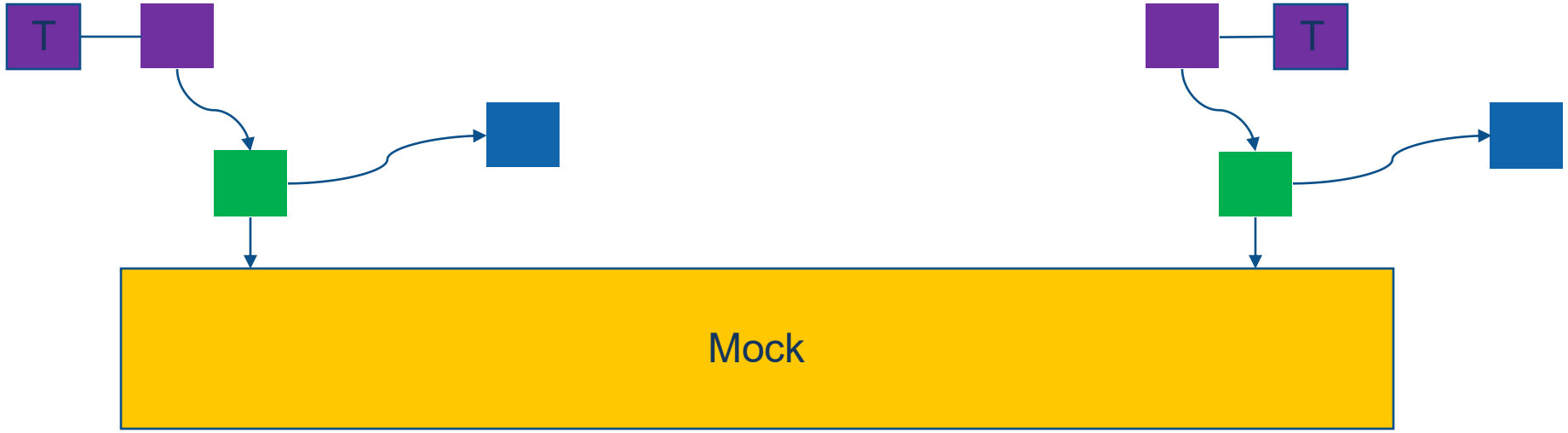


Testing



Testing transforms unknown unknowns into known knowns. Each test converts a dangerous assumption into a verified guarantee.

Testing



A mock is a simulated version of a software component that mimics the behavior of a real component for testing purposes.